Report Project Finding Similar Items

Silvia Guidi

March 2026

1 Introduction

The objective of this project is to implement a high-performance similarity detection pipeline for the arXiv dataset, focusing on the thematic overlap between scholarly abstracts. Given the scale of the corpus—approximately 2.97 million documents—a standard pairwise comparison is computationally infeasible.

Instead, this implementation utilizes a combination of MinHash and Locality Sensitive Hashing (LSH) to identify similar research papers within a 1.5-hour execution window.The system is engineered to operate under the 12.7GB RAM limit of the Google Colab Basic Tier. By leveraging Numba-accelerated JIT compilation and memory-mapped file storage, the pipeline achieves linear scalability, ensuring that the memory footprint remains stable even as the data volume increases thirtyfold during the transition from benchmarks to the full archive. This report details the normalization, indexing, and validation strategies used to maintain a target similarity threshold of 0.55 across the entire repository

2 The dataset

The project centers on the arXiv Dataset, a repository of scholarly articles spanning decades of scientific advancement. While each entry includes authors, categories, and titles, this implementation focuses exclusively on the abstract and the document ID. The abstract serves as the semantic "fingerprint" of the research, while the ID ensures traceability back to the original source. By stripping away peripheral metadata, the data are organized into a lightweight structure, ensuring that computational power is spent entirely on analyzing the thematic content of the papers.

3 Pre-processing

3.1 Data Cleaning

To prepare the arXiv abstracts for the algorithm, the code employs a high-performance normalization pipeline. This stage is critical because raw scien-

tific text contains "noise", such as varying capitalization and punctuation, that would otherwise cause the algorithm to treat identical sentences as different. The core of this cleaning process relies on the *string* package, a built-in Python module. Specifically, it provides a pre-defined constant called "punctuation", which is a single string containing every standard ASCII symbol. By using this set of characters as a reference, a translation table is constructed with the aim of mapping every punctuation mark to a "null" value, allowing them to be stripped away in a single, lightning-fast pass. This approach utilizes the "str.maketrans" function to create a pre-computed lookup map in memory. Rather than forcing the computer to perform a logical "if-then" check for every individual comma or period, the system uses each character's internal numerical code to jump directly to a "delete" instruction within the table. This bypasses the typical overhead of high-level string search, moving to optimized, low-level operations.

3.2 Shingling

Once the abstracts have been cleaned, they are converted into a set of numerical representations through a process known as k-shingling. Theoretically, a k-shingle is a contiguous subsequence of tokens (in this case, characters) of length k found within a document. The primary objective of shingling is to preserve the local structure of the text.

Selecting the value of k is a critical balancing act. If k is too small (e.g., $k = 3$), almost every scientific abstract would appear similar because of common short sequences, if k is too large, even a single character difference or a typo would render two otherwise identical shingles completely different, making the system too rigid. Empirically, $k = 9$ is the safe choice for scientific texts: it is long enough to capture unique technical words, but short enough to ignore minor typos or slight changes in wording. By choosing this specific length, the system can tell the difference between two papers about different topics even if they use the same basic academic phrases. This ensures that when the computer finds a match, it is because the actual research ideas are similar, not just the grammar.

The technical implementation transforms these shingles into a searchable mathematical format by processing each 9-character string through the MurmurHash3 (mmh3) function. This hash algorithm generates a numerical identifier for each sequence, specifically chosen for its uniform distribution to minimize collisions. To maintain consistency and computational efficiency, a bitwise "& 0xFFFFFFFF" mask is applied to every output, constraining each value to a standardized 32-bit unsigned integer range.

Finally, rather than storing these results in a standard Python list, they are packed into a NumPy array. By converting memory-heavy text abstracts into a compact NumPy array of 32-bit integers, the system reduces the data footprint required for analysis. This transition from raw characters to a structured numerical format allows the system to store and process document data with significantly higher spatial efficiency than raw text.

4 The Algorithms

4.1 The MinHash

After the abstracts are transformed into sets of shingle hashes, comparing the similarity of thousands of documents using their complete shingle sets creates significant computational overhead in terms of memory and execution time. To mitigate this, the MinHash algorithm is used to provide a compressed representation of each document.

MinHash is a technique designed to estimate the Jaccard Similarity without actually comparing every element. It works by "compressing" a large, variable-size set of shingles into a small, fixed-size numerical "signature." In this project, the shingles are compressed into just 100 integers per document. This allows to estimate how similar two papers are simply by counting how many of these 100 numbers match, drastically reducing the complexity of the task.

To generate these signatures, the code uses a mathematical trick that involves random permutations. For each of the 100 slots in the signature, it is applied a unique hash function defined as

$$h(x) = (ax + b)(modp)$$

where a and b are random coefficients, and p is a large prime number (2^{32}). The 'minhash_core' function iterates through every shingle in an abstract, applies all 100 hash functions, and keeps only the minimum value found for each one.

Processing a signature matrix with 100 hash functions across thousands of documents requires millions of operations, which typically creates a significant computational bottleneck in Python. This implementation resolves these performance constraints through three technical strategies:

1. Just-In-Time (JIT) Compilation: The minhash_core function is optimized using the @njit decorator from the Numba library. This instruction triggers a pre-execution compilation step that translates the Python-based mathematical loops into machine-level code. By bypassing the standard Python interpreter, the system executes the hashing and comparison logic at hardware-native speeds.

2. Parallel Processing: the system utilizes a ProcessPoolExecutor to distribute the workload across multiple CPU cores simultaneously. Instead of calculating signatures for one abstract at a time, the minhash function maps the signature task across a pool of workers. This concurrent execution reduces the total runtime proportionally to the number of available processing cores.

3. Memory Mapping (np.memmap): to prevent the signature matrix from exhausting available RAM, the implementation uses np.memmap to create a binary file on disk that behaves like an in-memory NumPy array. This allows the system to update the signature matrix directly to storage, enabling the processing of massive datasets that exceed the physical memory capacity of the computer.

4.2 Locality Sensitive Hashing (LSH)

Even with fast MinHash signatures, finding every similar pairs in a set of documents would take an impractical amount of time. Locality Sensitive Hashing (LSH) is the algorithmic "shortcut" that solves this by focusing only on pairs that are likely to be similar.

The core idea of LSH is to hash the signatures into buckets such that similar documents end up in the same bucket with high probability. In the code, this is achieved by dividing the 100-row signature matrix into $b = 20$ bands, each containing $r = 5$ rows. For each band, the system takes the r rows for a document, converts them into a byte-string, and uses this as a key in a hash table. If two documents have the exact same values for those specific r rows, they will hash to the same key and be placed in the same bucket. Instead of comparing every paper against every other paper, we only compare those that share a bucket. This effectively "prunes" the search space, discarding millions of pairs that are obviously different and focusing our computational power only on the most promising matches.

The selection of b and r is the most delicate tuning step in the LSH algorithm because it defines the probability that two documents with a certain Jaccard Similarity will be identified as candidates. Mathematically, this probability is expressed by the S-curve formula:

$$P(candidate) = 1 - (1 - s^r)^b$$

and graphically represented in figure 1.

This function creates a characteristic threshold that clearly divides the results: documents with a similarity above a certain critical point have high probability of being identified (high recall), while those below it are almost entirely filtered out (high precision).

By configuring the parameters to 20 bands (b) and 5 rows (r), the system establishes a mathematical "tipping point" or threshold (t) where the probability of a pair being flagged as a candidate is exactly 50%. This threshold is estimated by the formula:

$$t \approx \left(\frac{1}{b}\right)^{1/r}$$

By substituting our values, $t \approx (1/20)^{1/5} \approx 0.549$. This result confirms that the target of 0.55 serves as the intended "tipping point" for the algorithm.This specific balance ensures high recall by giving similar papers 20 different chances to be flagged as a match, while maintaining precision by requiring a significant sequence of 5 identical signature values to trigger a collision. Ultimately, this S-curve approach allows the algorithm to filter out millions of unrelated documents and focus only on abstracts with a meaningful conceptual overlap of 55% or higher.

To validate the effectiveness of the LSH shortcut, the final stage of the pipeline calculates the Empirical Precision. Since LSH is a probabilistic method, the precision function acts as a quality control mechanism by testing a random

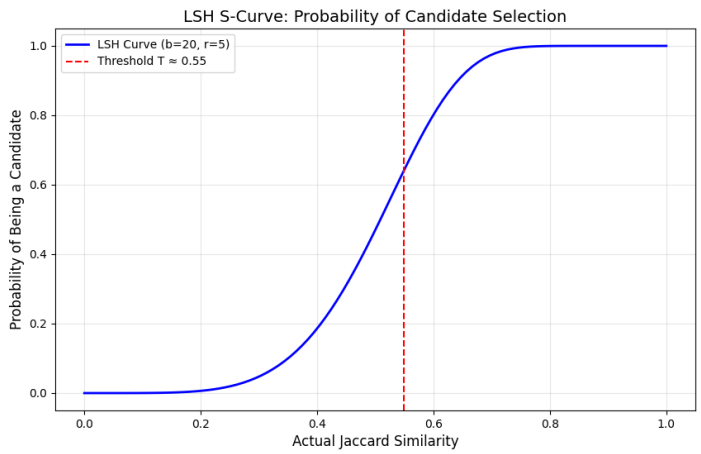

Figure 1: This graph illustrates the probability that a pair of abstracts will be identified as candidates based on their actual Jaccard Similarity. The steep "S" shape demonstrates the algorithm's efficiency: it creates a sharp transition at a similarity threshold of approximately 0.55.

subset of the identified candidates. Instead of verifying every single pair—which would be counterproductive to speed goals—the function draws a representative sample (defaulting to 100 pairs) and computes their actual similarity using the full signature matrix. A pair is classified as a True Positive if the mean agreement between their 100 MinHash signatures meets or exceeds the set threshold of 0.55. The function then divides the number of true positives by the total sample size to provide a percentage that represents how "reliable" the buckets were. This empirical result is crucial for tuning: if the precision is too low, it suggests the buckets are too broad and catching too much "noise," whereas a very high precision confirms that the $b = 20, r = 5$ configuration is successfully isolating truly similar research abstracts.

5 Results and Scalability Analysis

The implementation provides two distinct execution modes controlled by the 'USE_WHOLE_DATA' flag, allowing for either a comprehensive search across the entire corpus or a targeted investigation into the algorithm's performance dynamics.

When the system is configured to run the full dataset, it employs aggressive memory management strategies to handle over two million documents. This includes the use of gc.collect() to clear unreferenced objects and the deletion of the raw abstract arrays once the MinHash signatures are finalized. By saving the resulting signature matrix as a .npy checkpoint, the pipeline ensures that

the heavy computational work of shingling and hashing does not need to be repeated for different LSH parameter tests.

To rigorously evaluate how the system handles increasing volumes of data, a scalability benchmark was implemented using document subsets of 10 000, 25 000, 50 000, and 100 000 records. This approach allows to observe the relationship between data size (n) and execution time.

By capturing the minhash, lsh, and total execution times for each size increment, the system generates a performance profile. The empirical precision is also tracked at each step to ensure that the gain in speed does not come at the cost of accuracy, maintaining a reliable threshold of approximately 55% across different data volumes.

5.1 Scalability Analysis

The primary challenge in similarity detection at scale is transitioning from a theoretical model to an implementation capable of handling millions of records without exponential growth in resource consumption. While the previous sections established the mathematical foundations of MinHash and LSH, this section evaluates the pipeline's performance under increasing data loads. By benchmarking subsets ranging from 10,000 to 100,000 documents before executing the full corpus, we aim to verify that the system maintains linear complexity ($O(n)$) in both execution time and memory footprint. This empirical analysis demonstrates the efficiency of the chosen parameters and confirms the robustness of the implementation within the hardware constraints of the Google Colab environment.

5.1.1 Temporal Scalability and Complexity Analysis

The results of the scalability benchmark demonstrate the high efficiency and robust performance of the combined MinHash and LSH pipeline when applied to the arXiv dataset. By testing subset sizes ranging from 10 000 to 100 000 papers, the algorithm's behavior reveals clear trends in both computational speed and result quality.

The benchmark results represented in Figure 2 highlight a distinct contrast between the linear growth of the preprocessing stage and the exceptional speed of the candidate selection phase. As expected, the time required for shingling and MinHash signature generation grows linearly with the number of papers; processing 10 000 papers took approximately 19.94s, while 100 000 papers required 177.19s (3 mins). This consistent linear progression confirms that the ProcessPoolExecutor and Numba-accelerated core effectively manage the workload as the dataset scales.

It is important to note that these performance metrics were recorded using the Basic Tier of Gemini, which provides access to 2 CPU cores. The implementation is designed to automatically detect and utilize the maximum number of available CPU cores on any given machine. Consequently, because the MinHash phase is highly parallelizable, the runtime is bound to decrease significantly if

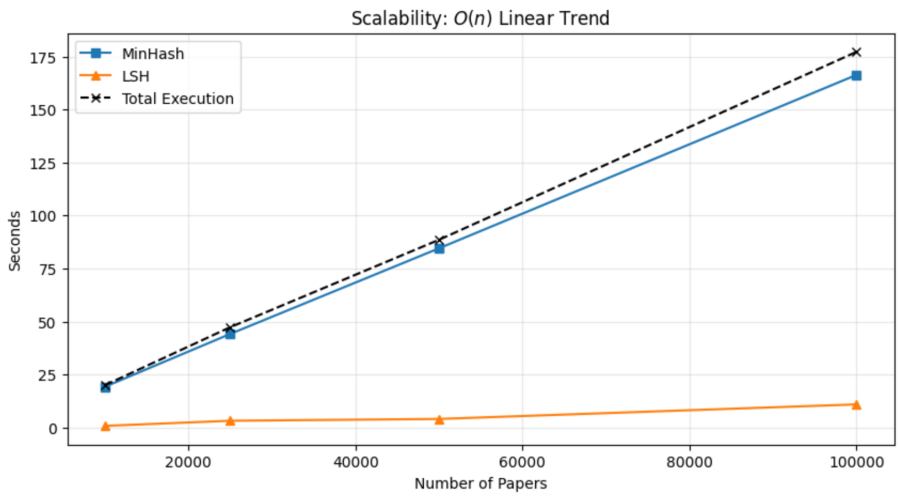

Figure 2: Figure 2: Scalability Analysis and $O(n)$ Linear Trend. This graph illustrates the execution time across increasing dataset sizes, from 10,000 to 100,000 papers

the pipeline is executed on hardware with higher core counts.This efficiency is further proven by the full dataset pipeline: for 2,968,861 documents, even on this limited hardware, the MinHash phase was completed in approximately 1.5 hours.The LSH phase remains remarkably fast even as the dataset size increases. While it took only 1.36s for the smallest subset and 11.22s for 100 000 papers, the jump to the full 3-million-document corpus only required roughly 5 minutes to identify 113 982 candidate pairs. This confirms that the banding and bucketing strategy successfully bypasses the $O(n^2)$ complexity, turning what would be a multi-year brute-force comparison into a task that finishes in minutes.

5.1.2 Spatial Scalability and Memory Efficiency

Another critical requirement for processing the arXiv dataset is maintaining a stable memory footprint, especially when scaling to nearly 3 million documents. The system was developed and tested using the Basic Tier of Google Colab, which imposes a strict hardware constraint of 12.7GB of RAM. To prevent the kernel from crashing due to memory exhaustion—a common risk when handling millions of records—the implementation employs aggressive memory management and disk-writing strategies.

During the execution of the full pipeline (2.97M documents), system monitoring indicated that RAM usage stabilized at approximately 9GB during the MinHash generation phase. This plateau is achieved by processing document signatures and writing results directly to a binary file on disk using memory mapping, rather than storing the entire signature matrix in physical RAM. Once the MinHash phase concludes, the memory usage decreases rapidly as the

raw abstract arrays are deleted and unreferenced objects are cleared.

The efficiency of this approach is evidenced by the comparison between the scalability benchmarks and the full-scale run. While the dataset size increased by a factor of 30, the RAM footprint only grew from under 3GB to 9GB. This sub-linear growth in memory consumption ensures that the implementation remains robust even under the 12GB ceiling of standard cloud environments, proving that high-performance similarity search is possible without the need for specialized, high-memory instances.

5.2 Precision

Throughout the scaling process, the Empirical Precision remained consistently high, generally fluctuating between 64% and 68% for smaller subsets. Even when scaling to the full dataset of nearly 3 million papers, the Empirical Precision stood at 63.00%, still exceeding the target threshold of 55%.This indicates that the chosen parameters ($b = 20, r = 5$) are highly effective at isolating truly similar abstracts across the entire arXiv archive. The "S-Curve" logic ensures that while millions of unrelated pairs are discarded instantly, the candidates that do collide in the LSH buckets have a high probability of being genuine research "neighbors," even when the total number of documents increases by a factor of 300.

6 Conclusion

The implementation of the MinHash and LSH pipeline has successfully demonstrated that identifying similar items in a massive corpus like arXiv is achievable without the need for high-end supercomputing clusters. By focusing on the semantic content of abstracts and utilizing Numba-accelerated JIT compilation, the system achieved a hardware-native execution speed that processed nearly 3 million documents in approximately 1.5 hours on a dual-core machine.

The experimental results validate the strategic choice of LSH parameters ($b = 20, r = 5$). The resulting S-curve created a sharp inflection point at a 55% similarity threshold, effectively filtering out noise while identifying over 113 000 candidate pairs from the full dataset. Furthermore, the empirical precision remained robust, hovering over 60% even at the largest scale, confirming that the probabilistic nature of LSH does not sacrifice the reliability of the matches found.

Ultimately, this project highlights the power of locality-sensitive algorithms in transforming $O(n^2)$ problems into manageable $O(n)$ tasks. The system is fully scalable; as more CPU cores are introduced, the linear growth of the MinHash phase will continue to decrease, making this an ideal architecture for real-time similarity search in large-scale academic and industrial knowledge bases.

I/We declare that this material, which I/We now submit for assessment, is entirely my/our own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my/our work, and including any code produced using generative AI systems. I/We understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study.